

## 1. Computer Vision

### 1.1 Digital images & sensors

An image (as 2D signal) is a continuous function. A pixel is a discrete sample of that cont. function.

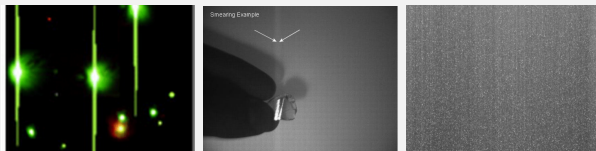
$$I : \{1, \dots, X\} \times \{1, \dots, Y\} \rightarrow S$$

Some challenges with images: transmission interference, compression artifacts, spilling, scratches, sensor noise, low contrast or resolution, motion blur.

#### Charge coupled device (CCD)

An array of photosites (a bucket of electrical charge) hold charge proportional to incident light intensity during exposure. Charges move down through sensor array, ADC (analog-digital-converter) measures line-by-line.

1. **Blooming:** Oversaturated photosites cause vertical channels to "flood" (bright vertical line)
2. **Bleeding/Smearing:** While charge in transit, bright light hits photosites above - worse with shorter shutter times (electrical shutters only)
3. **Dark current:** CCDs produce thermally generated charge: random noise despite darkness. Avoid by cooling, worsens with age.



(1) (2) (3)

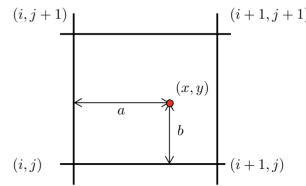
#### CMOS Sensors

Mostly same as CCD, but each sensor has amplifier. Cheaper, lower power, no blooming, on-chip integration with other components, but less sensitive and more noise. **Rolling shutter** is an issue for sequential read-out of lines when camera moving.  $\Rightarrow$  misalignment/distortions, **Dark current**

**Undersampling:** Causes loss of information and introduces **aliasing** (incorrect reconstruction, due to low sampling-rate ).

**Reconstruction:** Sample  $\mapsto$  cont. Signal. Needs assumption of behaviour.

**Bilinear Interpolation:** Reconstruction ( $\hat{f}$ ) for 2D sampling. Assumes linear behaviour.



$$\begin{aligned} \hat{f}(x, y) = & (1-a)(1-b) \cdot f(i, j) \\ & + a(1-b) \cdot f(i+1, j) \\ & + a b \cdot f(i+1, j+1) \\ & + (1-a) b \cdot f(i, j+1) \end{aligned}$$

#### Nyquist-Shannon Sampling Theorem

Sample freq. ( $\omega_s$ ), highest freq. ( $\omega_{max}$ ):  $\omega_s \geq 2\omega_{max}$

**Proof:**  $h \circ g, h = w$ .  $\omega_{max}$ ,  $g = \text{grid with spaces}$   
 $\frac{1}{\omega} \xrightarrow{CT} H \in [-\omega_{max}, \omega_{max}]$ ,  $G = GwS$ ,  $H * G \Rightarrow \dots$

**Geometric resolution:** # pixels per area

**Radiometric resolution:** # bits per pixel

**Image noise:**

**Additive Gaussian:**  $I(x, y) = f(x, y) + c$  where  $c \sim N(0, \sigma^2)$  (so density  $p(c) = (2\pi\sigma^2)^{-1} e^{-\frac{c^2}{2\sigma^2}}$ )

**Poisson (shot) noise:**  $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$ .

**Rician noise (MRI):**  $p(I) = \frac{1}{\sigma^2} \exp\left(-\frac{(I^2 + f^2)}{2\sigma^2}\right) I_0\left(\frac{If}{\sigma^2}\right)$

**Multiplicative noise:**  $I = f + fc$ .

**Signal to noise ratio (SNR):** Index of image quality:

$$s = \frac{F}{\sigma} \text{ where } F = \frac{1}{XY} \sum_{x=1}^X \sum_{y=1}^Y f(x, y).$$

**Peak SNR (PSNR):**  $s_{\text{peak}} = \frac{F_{\text{max}}}{\sigma}$ .

**Color camera concepts:** Prism (splitting light, 3 sensors, requires good alignment), Filter mosaic (coat filter on sensor, grid, introduces aliasing), Filter wheel (rotate filter in front of lens, static image), CMOS sensor (absorbing colors at different depths)

### 1.2 Image Segmentation

Partition image into regions of interest. e.g. foreground/background, motion segmentation, ...

**Complete segmentation of  $I$ :** regions  $R_1, \dots, R_N$  s.t.  $I = \bigcup_{i=1}^N R_i$  and  $R_i \cap R_j = \emptyset$  for  $i \neq j$ .

**Chromakeying:** Choose background (BG) color distinct from object (e.g. blue/green vs skin)  $c$ , then segmentation using BG subtraction with  $I_{BG} = c$ , Issues: hard  $\alpha$ -mask, variation not same in 3 channels

#### Thresholding

Produces binary image by labeling pixels *in* or *out* by comparing to some threshold  $T$  (RGB:  $T \in \mathbb{R}^3$ , Gray:  $T \in \mathbb{R}$ )  $B(x, y) = 1$  if  $\mathbf{I}_\alpha(x, y) \geq T$  else 0, finding  $T$  with trial and error, compare results with ground truth. Result:  $\mathbf{I}_{\text{proc}} = \mathbf{I}_\alpha \cdot \mathbf{I} + (1 - \mathbf{I}_\alpha) \cdot \mathbf{I}_{BG}$

**Distance Measures  $\mathbf{I}_\alpha$**

• **BG Subtraction:**  $\mathbf{I}_\alpha = |\mathbf{I} - \mathbf{I}_{BG}|$

• **Mahalanobis:**  $\mathbf{I}_\alpha = \sqrt{(\mathbf{I} - \mathbf{I}_{BG})^T \Sigma^{-1} (\mathbf{I} - \mathbf{I}_{BG})}$  where  $\Sigma$  is the BG image's covariance matrix:  $\Sigma_{ij} = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)]$ , estimate it from  $n \geq 3$  data points. to guarantee invert. of  $\Sigma$ , need 3 ind. samples:  $\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T, \mu := \frac{1}{n} \sum_{i=1}^n x_i$

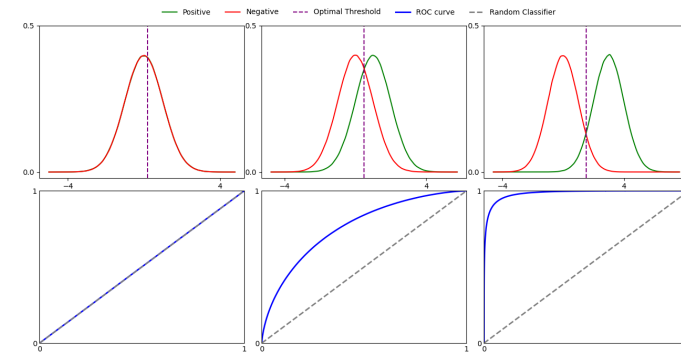
#### Receiver operating characteristic (ROC)

Describes performance of binary classifier (x: FPR y: TPR). With (table same as integral)

- true pos. (**TP**): # corr. classified FG,  $\int_\tau^\infty f_p dx$
- false pos. (**FP**): # BG wrongly clas. as FG,  $\int_\tau^\infty f_n$
- false neg. (**FN**): # FG wrongly clas. as BG,  $\int_0^\tau f_p$
- true neg. (**TN**): # corr. clas. BG,  $\int_0^\tau f_n dx$
- true pos. rate (**TPR**):  $\frac{TP}{TP+FN} = 1 - \text{FNR}$
- false pos. rate (**FPR**):  $\frac{FP}{FP+TN} = 1 - \text{TNR}$

**Trade-Off-Metric:**  $\beta = \frac{TN+FP}{TP+FN} \cdot \frac{V_{TN}+C_{FP}}{V_{TP}+C_{FN}}$ ,  $V$ : Val. corr. clas.,  $C$ : Cost for misclassifications

**Naming:** TPR=sensitivity, FPR=1-specificity,  $\text{FNR} = \frac{FN}{TP+FN}$ ,  $\text{TNR} = \frac{TN}{FP+TN}$ ,  $\text{Prec} = \frac{TP}{TP+FP}$

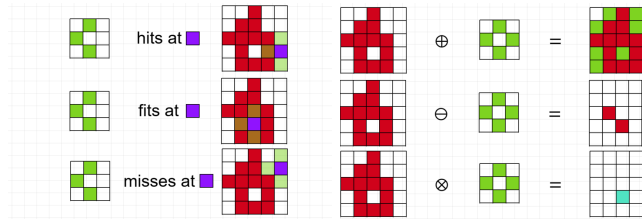


**Pixel neighbourhoods:** 4/8-connectivity: 4-neighb. means horiz. + vert. or 8-neighb. with diag.

**Connected Component Labelling (CCL):** Image processing technique to identify connected pixel regions.

## Region Growing

1. Select initial region/seed (manually or conservative thresholding)
2. Add neighbouring pixels if criteria satisfied
3. Repeat until no pixels can be added



It holds that:  $(I \odot S)^c = I^c \odot S$

## Skeleton and Medial Axis Transform (MAT)

Stick figure representations of region  $X \subset \mathbb{R}^2$ . Skeleton: union of centres of max. discs within  $X$ .

$$B = \{(2, 1), (2, 1), (2, 2), (2, 3), (3, 2)\} \hat{=} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

$S_{n(X)} = (X \ominus_n B) \setminus ((X \ominus_n B) \circ B)$  is the  $n$ -th skeleton subset. Skeleton is:  $S(x) = \bigcup_{n=1}^{\infty} S_n(x)$  Reconstr.:  $X = \bigcup_{n=0}^{\infty} S_n(X) \oplus_n B$  Used in object / char. recogn. Sensitive to noise, def. on digital grid poorly defined. Seq. thin. sometimes referred.

## 1.3 Image Filtering

**Shift-invariant:** same for all pixels,  $K$  not dep. on  $x, y$

**Linear:** Lin. comb. of nhb.  $\alpha L(I_1) + \beta L(I_2) = L(\alpha I_1 + \beta I_2)$  Mult. with  $I(x, y) \Rightarrow$  lin. Can be written  $K(i, j) = ai + bj + c$ . If  $2^{K(i, j)}$  is separable.

**Shift:**  $K = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \Rightarrow$  conv. shifts to right, shift invariant

## Correlation $K \circ I$

$$I'(x, y) = \sum_{(i, j) \in N(x, y)} K(i, j) I(x + i, y + j)$$

## Convolution $K * I$

$$I'(x, y) = \sum_{(i, j) \in N(x, y)} K(i, j) I(x - i, y - j)$$

Lin., ass., shift-invariant, comm. (if dims are same).  
For cont.:  $g(x) = f(x) * k(x) = \int_{\mathbb{R}} f(\tau) k(x - \tau) d\tau$

$K * I = K' \circ I$  where:  $K(i, j) = K'(-i, -j)$

**Padding:** clip to black, wrap around, copy edge, reflect across edge, vary filter!

**Separable:**  $K(m, n) \in \mathbb{R}^{m \times m} = f(m)g(n)$  more eff. apply first  $f \in \mathbb{R}^{m \times 1}$ , then  $g \in \mathbb{R}^{1 \times m}$  on image  $\in \mathbb{R}^{h \times w}$   
 $\mathcal{O}(w \cdot h \cdot m^2) \mapsto \mathcal{O}(w \cdot h \cdot m)$

**Gaussian:**  $G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ , rot. symmetric, single lobe, FT is again a Gaussian, separable, easily made efficient. Used to smooth, reduce noise, padding w. 0 or

clamping. **Prove low-pass**  $\mathcal{F}(G_{\sigma}) \circ \mathcal{F}(I) \stackrel{\text{CT}}{=} G_{\sigma} \circ \mathcal{F}(I)$   
High freq. mult. with small number, **Idea** weigh by dist.

## Kernel Examples $K_x = K_y^T$

| Low-pass                                                                        | High-pass                                                                   | Laplacian                                                            | Prewitt(x)                                                             |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------|
| $\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$ | $\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$ | $\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$ | $\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$ |
| Sobel(x)                                                                        | Id                                                                          | F-Diff.(x)                                                           | C-Diff.(x)                                                             |
| $\begin{pmatrix} 1 & 0 & -1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$          | $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$         | $\begin{pmatrix} -1 & 1 \end{pmatrix}$                               | $\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}$                             |

**Image sharpening:** enhances edges by increasing high frequency components:  $I' = I + \alpha |k * I|$  where  $k$  highpass filter,  $\alpha \in [0, 1]$ .

**Partial Derivatives:** Can be approximated if  $I$  is discrete by convolution with different Kernels.

$$\frac{\partial f}{\partial x} = \lim_{\epsilon \rightarrow 0} \left( \frac{f(x+\epsilon, y) - f(x, y)}{\epsilon} \right) \approx \frac{f(x_{n+1}, y) - f(x_n, y)}{\Delta x}$$

Larger = Better due: nghb affected by freq noise.  $\Rightarrow$  mix deriv. filter with low pass by extending filter

- **Laplacian:** Edges all dir,  $\nabla^2$ -appr. noise-sensitive, requires pre-smoothing. Zero-crossing for edge det.
- **Prewitt:** Edges in x,y-dir. More sensitive to noise than Sobel.
- **Sobel:** Smooths via central weights, better edge det than Prewitt. May require thresholding
- **F-Diff:** Compute first-order derivatives;  $\nabla$ -appr.
- **C-Diff:** More accurate than forward diff.

$$\text{Here: } \nabla^2 f(x, y) = \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2}$$

## 1.4 Edge Detection

### Laplacian operator

Find 0 s in 2nd deriv  $I''$  to locate edges. Rot. invariant, yields very noisy but thin and uninterrupted edges. Sensitive, blur first (Laplacian of Gaussian) or suppress edges with low gradient magnitude.

### Laplacian of Gaussian (LoG)

Blurring of image with Gaussian and Laplacian operator can be combined into convolution with LoG:

$$\text{LoG}(x, y) = -\frac{1}{\pi\sigma^4} \left( 1 - \frac{x^2 + y^2}{2\sigma^2} \right) \exp \left( -\frac{x^2 + y^2}{2\sigma^2} \right)$$

## Criteria

- **Grey-Level-Thresholding:** Apply simple thresholding
- **Advanced:** Compute std-deviation  $\sigma$ , mean  $\mu$  of graylevels in region and check  $(I(x, y) - \mu)^2 < (n\sigma)^2$

## Snakes

Active contour, polygon, where each point on contour moves away from seed while inclusion criteria in neighborhood is met, often smoothness constraint. Iteratively minimize energy:  $E = E_{\text{tension}} + E_{\text{stiffness}} + E_{\text{image}}$

**Markov Random fields:** Cost to assign label to each pixel, cost to assign pair of labels to connected pixels. Solve with graph cuts, source = FG, sink = BG. Minimize energy in polynomial time using MinCut. To get decent results, we need alpha estimation / border matting along edges.

## Morphological Operations

Op that applies structuring element  $S$  to the neighborhood of each pixel in  $I$ . Sets with x-y-coords. of 1s.

- **Dilation of  $I$ :**  $D = I \oplus S$ , apply  $D(p)$  to all pixels
- **Erosion of  $I$ :**  $E = I \ominus S$ , apply  $E(p)$  to all pixels
- **Hit-and-Miss:**  $H = I \otimes S$  exact match of  $I$  by  $S$
- **Opening:**  $O = I \circ S = (I \ominus S) \oplus S$
- **Closing:**  $C = I \bullet S = (I \oplus S) \ominus S$
- **Thinning:**  $I \oslash S = I \setminus (I \otimes S)$
- **Tickening:**  $I \odot S = I \cup (I \otimes S)$

## Definitions:

- $S$  fits  $I$  at  $p$  if  $\{y : y = p + s, s \in S\} \subset I$
- $S$  hits  $I$  at  $p$  if  $\{y : y = p - s, s \in S\} \cap I \neq \emptyset$
- $S$  misses  $I$  at  $p$  if:  $\{y : y = p - s, s \in S\} \cap I = \emptyset$
- **Dilation at  $p$ :**  $D(p) = 1$  if  $S$  hits  $I$  at  $p$  else 0
- **Erosion at  $p$ :**  $E(p) = 1$  if  $S$  fits  $I$  at  $p$  else 0

### Canny Edge Detector

Thin, interrupted edges, extended more completely than with simple thresh.

1. **Smoothing image:** with Gaussian filter
2. **Compute gradient, magnitude & orientation :** Use Sobel, Prewitt, ...  $M(x,y) = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$ ,  $\alpha(x,y) = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$
3. **Nonmaxima suppression:** quantize edges normal to 4 dirs, if smaller than either neighb. suppress
4. **Double thresholding:** Classify pixels as *strong*-, *weak*- or *non-edges*:  $E_{strong} \geq \theta_{high} > E_{weak} \geq \theta_{low}$
5. **Edge Tracking (Hysteresis):** Classify  $E_{weak}$  that are not 8-conn. to  $E_{strong}$  as *non-edge*.

### Hough Transformation

Maps cart. coords.  $(x,y)$  to parameter Space (p.S) to detect specific form.

1. **Subdivide p.S.:** Into discrete bins init to 0.
2. **Vote in p.s.:** For each edge-pixel in  $(x,y)$  compute potential points, map onto p.s.
3. **Detect peaks in p.s.:** Corresponds to form in  $(x,y)$

- **Lines (slope) :** Choose  $m \in [a,b]$ ; compute  $c = y - mx$ ; Plot  $(m,c)$ ; has  $\infty$  slope problem.
- **Lines (polar) :** Choose  $\theta \in [0,\pi]$ ; Compute  $\rho = x \cos \theta + y \sin \theta$ ; Plot  $(\rho,\theta)$
- **Circles:** Choose  $\theta \in [0,2\pi], r \in [0,c]$ ; Compute  $a = x - r \cos \theta, b = y - r \sin \theta$ ; Plot  $(a,b,r)$
- **Parabola:** Choose  $a,b \in \mathbb{R}$ ; Compute  $c = y - ax - bx^2$

**Corner detection:** Properties: acc. local., invar. against shift, rot., scale, brightness, noise robust, repeatability. Corners important since edges only well localized in 1 direction.

**Local Displacement Sensitivity:** Appr. change in intensity  $f$  for small shifts

$$S(\Delta x, \Delta y) = \sum_{(x,y) \in \text{window}} (f(x,y) - f(x + \Delta x, y + \Delta y))^2.$$

Using the linear approx.  $f(x + \Delta x, y + \Delta y) \approx f(x,y) + f_x(x,y)\Delta x + f_y(x,y)\Delta y$  we get

$$S(\Delta x, \Delta y) \approx \sum_{(x,y) \in \text{window}} [f_x(x,y)\Delta x + f_y(x,y)\Delta y]^2.$$

We write  $S(\Delta x, \Delta y) = (\Delta x \Delta y) \mathbf{M} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \Delta^T \mathbf{M} \Delta$  with

$$\mathbf{M} = \sum_{(x,y) \in \text{window}} \begin{pmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{pmatrix}$$

**Extracting Feature points:**  $\lambda_1, \lambda_2$  of  $\mathbf{M}$  we can detect:

- **Corner:**  $\lambda_1, \lambda_2$  large, same as  $\min_{\|\Delta\|=1} \Delta^T \mathbf{M} \Delta$
- **Edge:**  $\lambda_1 \gg \lambda_2$  or  $\lambda_2 \gg \lambda_1$
- **Flat Region:**  $\lambda_1, \lambda_2$  small

### Harris Corner Detection

Avoids EV computation. Uses appr.

1. **Compute M:**
2. **Compute C :**  $C(x,y) = \det(\mathbf{M}) - k \cdot (\text{trace}(\mathbf{M}))^2 = \lambda_1 \lambda_2 - k$  with  $k \in [0.04, 0.06]$ .
3. **Classify:** corner ( $C > 0$ ), edge ( $C < 0$ ) or flat region ( $|C|$  small)
4. **Nonmax suppression:** On corners

Properties:

- Invariant to shift:  $\nabla$  only shifted
- Inv. to rot: EV stay, eigenvectors rotate
- Inv. to brightness offset:  $\nabla(I + c) = \nabla(I)$
- not Inv to scaling:  $(\Delta_z, \Delta_y)$  is constant size: corner  $>$  wdw-size  $\Rightarrow$  edges

### Lowe's Scale Invariant Feature Transform (SIFT)

Used to track feature over multiple images.

1. **Create scale space:** Build scale space by using DoG for different  $k$ .
2. **Keypoint localization:** Find keypoints, which are extremas by considering neighborhood of current image (8 px), neighborhood of image one scale up and down (9 px each).
3. **Orientation assignment:** Compute gradient of neighborhood of keypoints,  $m(x,y) = \sqrt{A^2 + B^2}$ ,  $\theta(x,y) = \tan^{-1}(B/A)$  where  $A = (L(x+1,y) - L(x-1,y))$   $B = (L(x,y+1) - L(x,y-1))$  and  $L$  is the Gaussian-smoothed image at the keypoint's scale. Build histograms with 36 bins over 360 deg. Assign peak of the histogram as orientation to the keypoint
4. **Keypoint descriptor generation:** Take  $16 \times 16$  neighborhood around keypoint, divide into  $4 \times 4$  subblocks and compute an 8-bin orientation histogram based on the gradient directions. Results in 128 feature vector per keypoint.

**Enhance invariance to illumination:** Take resulting feature vector and normalise it. If illumination is non-linear threshold unit vector to 0.2 and then normalise again.

**Difference of Gaussian (DoG):** Appr. LoG, good in detecting edges, blob-like structures,

$DoG(x,y) = \frac{1}{k} e^{-\frac{x^2+y^2}{(k\sigma)^2}} - e^{-\frac{x^2+y^2}{\sigma^2}}$  Same as difference of two Gaussians with scale  $\sigma$  and  $k\sigma$ .

### 1.5 Fourier Transform

#### Discrete Fourier Transform

Represents signal in new basis: Transforming from spatial domain  $f$  to freq. domain  $F$  by:

$$F(I)(u,v) = \iint_{R^2} I(x,y) e^{-i2\pi(ux+vy)} dx dy$$

**Inverse:**  $I(x,y) = \iint_{R^2} F(I)(u,v) e^{i2\pi(ux+vy)} du dv$

**Discrete FT:**  $F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} I(x,y)$

$\cdot e^{-i2\pi(\frac{ux}{N} + \frac{vy}{M})}$ , where  $N \times M$  is the image size,  $I(x,y)$  the image value at pixel  $(x,y)$  and  $F(u,v)$  the Fourier coefficient at coordinates  $(u,v)$ . Can write as  $F = \mathbf{U} f$ , where  $F, f \in \mathbb{C}^{N \cdot M}$ ,  $\mathbf{U} \in \mathbb{C}^{N \cdot M \times N \cdot M}$  dim: in = out

**Def:**  $\cos(x) = \frac{e^{ix} + e^{-ix}}{2}$   $\sin(x) = \frac{e^{ix} - e^{-ix}}{2}$

**Dirac delta:**  $\delta(x) = 0$  if  $x \neq 0$  else undefined with  $\int_{-\infty}^{\infty} \delta(x) dx = 1$

**Sampling:** Mult with seq. of  $\delta$ -fnts

$F(\delta(x - x_0))(u) = e^{-i2\pi u x_0}$ . and  $\text{sinc}(u) = \frac{\sin(u)}{u}$

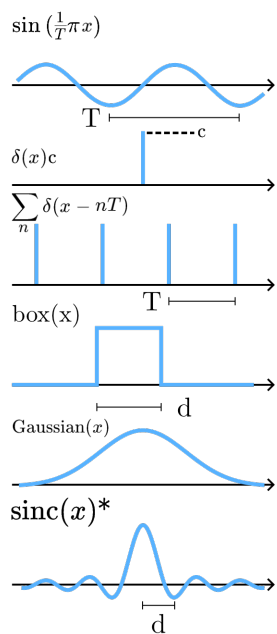
| Property    | $f(x)$                         | $F(u)$                                 |
|-------------|--------------------------------|----------------------------------------|
| Linearity   | $\alpha f_1(x) + \beta f_2(x)$ | $\alpha F_1(u) + \beta F_2(u)$         |
| Duality     | $F(x)$                         | $f(-u)$                                |
| Convolut.   | $(f * g)(x)$                   | $F(u) \circ G(u)$                      |
| Product     | $f(x)g(x)$                     | $(F * G)(u)$                           |
| Timeshift   | $f(x - x_0)$                   | $e^{-2\pi i u x_0} \cdot F(u)$         |
| Freq. shift | $e^{2\pi i u_0 x} f(x)$        | $F(u - u_0)$                           |
| Different.  | $\frac{dn}{dx^n} f(x)$         | $\left(\frac{i}{2\pi} u\right)^n F(u)$ |
| Multiplic.  | $xf(x)$                        | $\frac{i}{2\pi} \frac{d}{du} F(u)$     |

**Element-Wise Multiplication:**  $\circ$

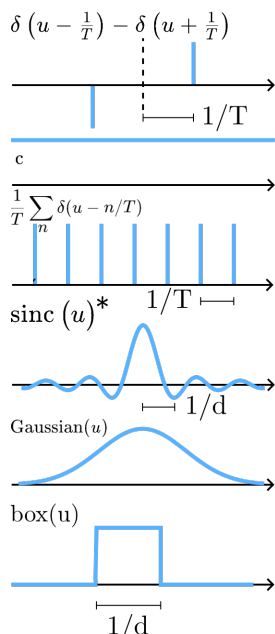
### Image Restoration

Image degradation is applying kernel  $h$  to some image  $f$ . The inverse  $\tilde{h}$  should compensate:  $f(x) \rightarrow h(x) \rightarrow g(x) \rightarrow \tilde{h}(x) \rightarrow f$ . Determine with  $F(\tilde{h})(u,v) = F(h)(u,v) = 1$ . Cancellation of freq., noise amplif. Regularize using  $\tilde{F}(\tilde{h})(u,v) = F(h) / (|F(h)|^2 + \varepsilon)$

## Spatial domain



## Frequency domain



can be done with SVD.

**Eigenfaces:** Eigenspace Matching for faces, struggle with lighting differences.

**Fisherfaces:** Improvement of EF by maximizing between-class scatter, minimizing within-class scatter.

Solution given by Eigenval. Dec:  $\mathbf{R}_B \mathbf{W} = \mathbf{R}_W \mathbf{W} \mathbf{A}$ .

$\text{rank}(\mathbf{R}_W) = C$ , where  $C$  amount of different classification classes. Transf. to  $C$ -dim subspace by PCA

**Find faces** on  $I$  with dataset of faces ( $d$ ): 1. PCA of  $d$ , 2. take patch  $I$ , 3. comp. reconstr. err. ( $e$ ), 4. repeat for each patch, 5. find loc. with min.  $e$ .

## Principal Component Analysis

1. *Normalize to remove brightness var.:*  $x'_i = \frac{x_i}{\|x_i\|}$
2. Center data by subtracting mean:  $x''_i = x'_i - \mu$ ,  $\mu = \frac{1}{N} \sum x'_i$
3. Compute covar. mat.:  $\Sigma = \frac{1}{N-1} \sum x''_i \cdot (x''_i)^T$
4. Compute eigendecomp. of  $\Sigma$  by solving  $\Sigma e = \lambda e$  with e.g. SVD ( $\Sigma = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$ )
5. Define  $U_k$  as first  $k$  eigenval. of  $\Sigma$ ,  $U_k = (u_1 \dots u_k)$  directions with largest variance centered at mean, perpendicular
6. PCA  $(x_i) = U_k^T (x_i - \mu) = U_k^T \cdot x''_i$

To decompress, use  $\text{PCA}^{-1}(x_i) = U_k \cdot y_i + \mu$

**Applications:** Face detect. Data compr. Data vis.

**PCs:**  $n = \# \text{PCs}$ ,  $m = \text{dim samples}$ ,  $o = \# \text{samples}$ :

- $n \leq m$ : Can't have more than  $m$   $\perp$  vect.
- $n \leq o$ :  $\text{rank } \Sigma \leq m$

**Storage Space:**  $I \in \mathbb{R}^{a \times a}$ ,  $I_{\text{compr}} \in \mathbb{R}^K$  tot. space:  $a^2$  (mean)  $+ a^2 K (U_k) + nK$  (compr. Image)

## 1.7 JPG & JPEG

1. Convert RGB  $\rightarrow$  YUV (Y luminance / brightness, UV color / chrominance). Humans more sensitive to color, compress colors with chroma subsampling (e.g. color of upper left pixel for  $4 \times 4$  grid)
2. Split image into  $8 \times 8$  blocks for each YUV component, apply 2D DCT to it. 64 values, top left = low freq., bottom right = high freq. DCT: variant of DFT, fast implementation, only real values
3. Compress using int. division with weighted matrix (Quantization table), compress bottom-right. Zig-zag run length encoding followed by Huffman.

High compression (10 – 100x) and 24 -bit colors possible, but artifacts / incorrect colors / aliasing. Edges are softened because sharp edges require high freq. JPEG2000 improves by using Haar transform globally,

not just on  $8 \times 8$  blocks, on successively downsampled image (image pyramid)

**Image pyramids:** iter. applied approx. filter / downsampler. Gaussian pyramid, Laplacian is difference between 2 levels in Gaussian pyramid.

**Haar transform:** Scaling capture info. at diff. freq., Translate captured info. at diff. loc. Can be repr. by filter+downsample. Poor energy compaction.

## 1.8 Optical Flow

Apparent motion of brightness patterns. We set  $u = \frac{dx}{dt}$ ,  $v = \frac{dy}{dt}$ ,  $I_x = \frac{\partial I}{\partial x}$ ,  $I_y = \frac{\partial I}{\partial y}$ ,  $I_t = \frac{\partial I}{\partial t}$

**Brightness constancy:**

$$I(x, y, t) = I\left(x + \frac{dx}{dt} \delta t, y + \frac{dy}{dt} \delta t, t + \delta t\right) \stackrel{\text{Taylor}}{\approx}$$

$$I(x, y, t) + \frac{\partial I}{\partial x} \left(\frac{dx}{dt} \delta t\right) + \frac{\partial I}{\partial y} \left(\frac{dy}{dt} \delta t\right) + \frac{\partial I}{\partial t} (\delta t)$$

**Optical flow constraint:**  $\frac{dI}{dt} = \frac{\partial I}{\partial x} \frac{dx}{dt} + \frac{\partial I}{\partial y} \frac{dy}{dt} + \frac{\partial I}{\partial t} = 0$

**Aperture problem:** 2 unknowns for every pixel  $(u, v)$  but only one equation  $\Rightarrow \infty$  solutions (underdetermined SLE), opt. flow defines a line in  $(u, v)$  space, compute normal flow. Need additional constraints to solve.

## Horn & Schunck Algorithm

**Assumption:** values  $u(x, y)$ ,  $v(x, y)$  are smooth and change slowly with  $x, y$ .

Minimize  $e_s + \lambda e_c$  for  $\lambda > 0$  where  $e_s = \iint ((u_x^2 + x_y^2) + (v_x^2 + v_y^2)) dx dy$  (smooth.)

$e_c = \iint (I_x u + I_y v + I_t)^2 dx dy$  (bright. const.)

Coupled PDEs solved using iter. methods and finite diffs:  $\frac{\partial u}{\partial t} = \Delta u - \lambda (I_x u + I_y v + I_t) I_x$  and  $\frac{\partial v}{\partial t} = \Delta v - \lambda (I_x u + I_y v + I_t) I_y$ . Has errors at boundaries / information spreads from corner-type patterns.

## Lucas-Kanade

Assumption: neighb. in  $N \times M$  patch  $\Omega$  have same motion  $(uv)^\top$ , small mov., bright. const.  $E = \sum_{x,y \in \Omega} (I_x(x, y)u + I_y(x, y)v + I_t(x, y))^2$   $MU = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = -(\sum I_x I_t) \begin{pmatrix} u \\ v \end{pmatrix}$  (over patch  $\Omega$ ) First compute  $I_x, I_y, I_z$ . Let  $M = \sum (\nabla I)(\nabla I)^T$  and  $b = (-\sum I_x I_t - \sum I_y I_t)$  Then at each pixel, compute  $U$  by solving  $MU = b$  by using least squares  $U = M^{-1}b = (M^T M)^{-1} M^T b$ . If  $M$  singular / fails if all gradient vec. in same dir (along edge, smooth regions). Works with corners, textures.

\* Holds if  $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$  (normalized), for  $\frac{\sin(x)}{x}$  use  $\text{sinc}(\pi t) \rightarrow \text{Box}(u)$  and  $\text{sinc}(\pi t) \rightarrow \text{Box}(u)$

**Convolution Theorem:** Conv. in spatial domain = el. mult. in freq. domain and vice-versa:  $\mathcal{F}[f * g] = F \circ G$  and  $\mathcal{F}[f \cdot g] = F * G$ . Can be used to efficiently compute convolution:  $h * f = \mathcal{F}^{-1}[H \circ F]$ , with  $F = \mathcal{F}(\text{Image})$

- **Low-pass (Smoothen):**  $H_{\text{low}, \sigma} = \mathcal{F}(G_\sigma(x, y))$
- **High-pass (Sharpen):**  $H_{\text{high}, \sigma} = 1 - H_{\text{low}, \sigma}$
- **Band-pass:**  $H_{\text{band}, [\sigma_1, \sigma_2]} = H_{\text{low}, \sigma_1} - H_{\text{low}, \sigma_2}$

## 1.6 Unitary Transform

**Linear Operations:**  $O(\alpha x + \beta y) = \alpha O(x) + \beta O(y)$   $\alpha, \beta \in \mathbb{R}$ . Alt. Matrix **O**. For pictures  $I \in \mathbb{R}^{M_1 \times M_2}$ , with  $N = M_1 \times M_2$  we have  $c = \mathbf{O}f$ ,  $c, f \in \mathbb{R}^N$ ,  $\mathbf{O}^{N \times N}$

**Unitary Operations:** Lin. Op. with  $\mathbf{O}^{-1} = \mathbf{O}^H$ . Preserves Norm/Energy:  $\|c\|^2 = f^H \mathbf{A}^H \mathbf{A} f = \|f\|^2$

**Auto-Correlation:**  $R_{ff} = E[f_i \cdot f_i^H] = \frac{F \cdot F^H}{n}$ .  $R_{ff}(i, j)$  denotes how pixel  $f_i$  correlates with pixel  $f_j$ .  $R_{ff} \in \mathbb{R}^N$ ,  $R_{cc} = E[cc^H] = E[Af f^H A^H] = A R_{ff} A^H$

**Eigenmatrix:**  $\Phi$  of autocorrelation matrix  $R_{ff}$ :  $\Phi$  is unitary, columns form set of eigenvectors of  $R_{ff}$ :

$R_{ff} \Phi = \Phi \Lambda$  where  $\Lambda$  is a diag. matrix of eigenvecs.  $R_{ff}$  is symmetric nonneg. definite, hence  $\lambda_i \geq 0$ , and normal:  $R_{ff}^H R_{ff} = R_{ff} R_{ff}^H$ .

**Eigenspace Matching:** Find similar image  $I_c$  to set of  $n$  images  $I_i$ .  $\arg \min_i D_i = \|I_i - I_c\|$ . Autocorr. all options expensive. Preselect with  $\arg \min_i D_i = \|p_i - p_c\|$  where  $p = \mathbf{E}^T \hat{I}$  (Eigenval. Dec.),  $\hat{I} = I - \bar{I}$ ,  $\bar{I} = \frac{1}{n} \sum^n I_i$ . Same

**Iterative refinement:** Estimate OF with Lucas-Kanade. Warp image using estimate OF. Estimate OF using warped image. Refine by repeating and add up all estimates. **Gradient method** fails when intensity structure within window is poor, displacement large, inconstant light, sol: **CtFE**

**Coarse-to-fine Estimation:** Create levels by gradual subsampling. Start at coarsest level, estimate OF, iterate and add until reached finest level. Still fails if large lighting changes happen.

**Affine Motion:** Each pixel provides 1 lin. constr., 6 global unknowns. Solve LSE. From bright. const. eq.:  $I_x(a_1 + a_2x + a_3y) + I_y(a_4 + a_5x + a_6y) + I_t \approx 0$

**SSD tracking:** Large displacements, extract template around pixel, match within search area, use correlation, choose best match.

**Bayesian Optical flow:** Some low-level human motion illusions can be explained by adding an uncertainty model to LK. E.g. bright. const. with noise.

1.9 Video Compression

Visual perception < 24 Hz. Flicker > 60 Hz in periphery. **Bloch’s law:** If stimulus duration ≤ 100 ms, duration and brightness exchangeable. If brightness is halved, double duration. This enforces > 10 Hz for videos. **Interlaced video format:** 2 temporally shifted half images (in bands) increases frequency, reduces spatial resolution. Full image repr. is progressive.

Video Compression w/ Temporal Redundancy

Predict current frame based on previously coded frames. Introducing 3 types of coded frames:

1. I-frame: Intra-coded frame, coded independently

2. P-frame: Predictively-coded based on previously coded frames (e.g. motion vec. + changes)

3. B-frame: Bi-directionally predicted frame, based on both previous and future frames.

Inefficient for many scene changes or high motion.

**Motion-compensated prediction:** partition video into moving objects - generally pretty difficult. **Block-Matching Motion Est.:** partition each frame into blocks, describe motion of block / find best matching block in reference frame. No obj. det., good perf. Can be sped up with 3 step log search, and improve precision with half-pixel motions. We encode the residual error (with JPG). Motion vector field (set of motion vec. for

each block in frame) is easy to encode, fast, simple, periodic, practical. **MPEG Group of Pictures (GOP):** Starts with I-frame, ends with B- ("open") or P-frame ("closed")

1.10 CNN & Radon Transform

Given input  $x$ , learning target  $y$ , loss function  $\mathcal{L}$  compute kernel weights  $\theta$  using prediction  $f(x, \theta)$  :  $\arg \min_{\theta} \mathcal{L}(y, f(x, \theta))$ . Linear classifier  $f(x, \theta) = Wx + b$  where  $\theta = \{W, b\}$ . Use activation func.  $\phi$  (sigmoid, ReLU, tanh, ...) to introduce non-linearity. Use gradient descent and back propagation (recursive appl. of chain rule to compute gradients) to find  $\theta$ . Transformers, GANs, stable diffusion etc enable modern VC breakthroughs. **Classification:**  $f(x_1, \theta)$  as score, take class with larger score. With  $y_i \in \mathbb{N}, s_i = f(x_i, \theta)$ , we get loss function:

$$\mathcal{L}(y, f(x, \theta)) = - \sum_{i=1}^N \log \left( \frac{\exp(s_{i, y_i})}{\sum_j \exp(s_{i, j})} \right)$$

**Regression:**  $f(x_1, \theta)$  as value, can be used for classification by comparing value. With  $y_i \in \mathbb{R}^n, s_i = f(x_i, \theta)$ , we get:  $\mathcal{L}(y, f(x, \theta)) = \sum_{i=1}^N \|y_i - s_i\|^2$

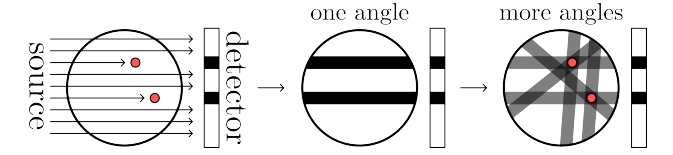
Radon transform

Given object with unknown density  $f(x, y)$ , find  $f$  by sending rays from all dirs through object and measure absorption on the other. side. Assume parallel beams for given angle and no spreading of beam. X-Ray along line  $L$  at distance  $s$  has intensity  $I(s)$ , travelling  $\partial s$  reduces intens. by  $\partial I$ . reduction depends on intens. and optical density  $u(s) : \frac{\partial I}{I(s)} = -u(s)\partial s$ . Radon transform of  $f(x, y) : Rf(L) = \int_L f(x)|dx|$ . With  $(x, y) = (\rho \cos \theta - s \sin \theta, \rho \sin \theta + s \cos \theta)$  we get:  $R(\rho, \theta) = \int u(x, y)ds$ . We now want to find  $u(x, y)$  given  $R(\rho, \theta)$ .

The continuous case of a radon transform of a line is:

$$R(\rho, \theta) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} u(x, y) \delta(\rho - x \cos \theta - y \sin \theta) dx dy$$

**Properties of RT:** Linear, shifting input shifts the RT, rotating input rotates RT, RT of 2 D convolution is 1D convolution of RT with respect to  $\rho (R(f *_{2D} g) = R(f) *_{1D} R(g), \text{ RHS with fixed } \theta)$



Back Projection

Given RT, find  $u(x, y)$  Fourier slice thm (apply with all proj. angles  $\theta$ ):

1. **Measure attenuation data:** Measured along lines parameterized by  $\rho$  = perp. distance from origin and  $\theta$  = the angle of proj.

$$R(\rho, \theta) = \int_{L(\rho, \theta)} f(x, y) ds$$

2. **1D FT of projection data:** Convert to freq. domain.  $\omega$  represents the frequency in the radial direction for the projection at angle  $\theta$ .

$$\hat{R}(\omega, \theta) = \mathcal{F}_{1D}\{R(\rho, \theta)\} = \int_{-\infty}^{\infty} R(\rho, \theta) e^{-i\omega\rho} d\rho$$

3. **2D inverse FT and sum with previous image (backpropagate):**

$$u(x, y) = \mathcal{F}_{2D}^{-1} \left\{ \sum_{\theta} \hat{R}(\omega, \theta) \right\}$$

Requires precise attn. meas., sensitive to noise, unstable, hard to imple., blurring in final image. Add high-pass filter in Fourier domain after 2nd step.

2. Computer Graphics

2.1 Graphics Pipeline

1. **Modeling transform:** From object to world-space

2. **Viewing transform:** From world to camera-space

3. **Primitive processing:** Output primitives from transformed vertices

4. **3D clipping:** Remove primitives outside frustum

5. **Projection to screen-space**

6. **Scan conversion:** Discretize continuous primitives, interpolate attributes at covered samples

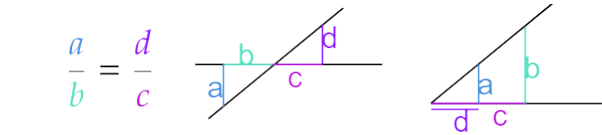
7. **Lighting, shading, texturing:** Compute colors

8. **Occlusion handling:** Update color (e.g. z-buffers)

9. **Display**

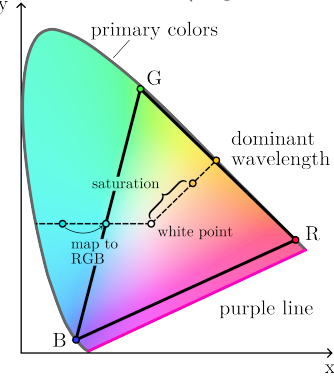
To achieve interactivity, every step has to be reversible. **Visualisation Pipeline:** Mapping thematic data (i.e. visualisation of statistics on a map) via attributed geometry into images. Has the following steps: Data Generation, Data Filtering, Visualisation Tools,

Rendering, Display. **Contemporary Pipeline:** CPU, Vector processing (pervertex ops, transforms, lighting flow control), Rasterization, Fragment processing (per-fragment ops, shading, texturing, blending), Display. **Intercept Theorem:**



2.2 Light & Colors

Light is mixture of many wavelengths. Consider  $P(\lambda)$  as intensity at wavelength  $\lambda$ . Humans project inf. dimens. to 3D color (RGB). CIE experiment: some colors are not comb. of RGB. (neg. red needed)



**Mixing Colors:** Mixing 3 lin. ind. primaries in the CIE-Chart: all colors inside/on spanning triangle creatable  
**IsoLine Sat.:** Parallel to round border:  
**Mac Adams Ellipses:** Quantifies how humans perceive diff. in color, not uniform across the spectrum.

**Color Spaces**

**RGB** (useful for displays, RGB colors specified)  
**CIE XYZ:** Change of basis to a neg. comp.  
**CIE xyY:** Chromaticity (color)  $(x, y)$  derived by normalizing XYZ color components:  $x = \frac{X}{X+Y+Z}, y = \frac{Y}{X+Y+Z}$ . Y is brightness.  
**CIE RGB:** (435.8, 546.1, 700.0 nm ). Linear combination span triangle, the Color Gamut.  
**CMY:** inverse (subtr.) to RGB.  $CMY = 1 - RGB$ .  
**YIQ:** Luminance Y, In-phase I (orange-blue), Quadrature Q (purple-green).  
 $\begin{pmatrix} Y \\ I \\ Q \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$   
**HSV:** hue (base color), saturation (purity of color), value / lightness / brightness (intuitive)  
**CIELAB / CIELUV:** Correct CIE chart colors to adjust for perceived "distance" betw. colors, nonlinear warp. MacAdams ellipses nearly circular.

**2.3 Transformations**  
**Homogeneous Coordinates:** Can represent affine maps (translation) with mat.-mul. Add dimension, project vertices  $(x\ y\ z)^T$  onto  $(\frac{x}{w}\ \frac{y}{w}\ \frac{z}{w}, w)^T = (xyzw)^T$ . Scale Invariant, vector space

Trans.  
 $\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Scale  
 $\begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rot(x)  
 $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi & 0 \\ 0 & \sin \phi & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rot(y)  
 $\begin{pmatrix} \cos \phi & 0 & \sin \phi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \phi & 0 & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rot(z)  
 $\begin{pmatrix} \cos \phi & -\sin \phi & 0 & 0 \\ \sin \phi & \cos \phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Shear(x,y,z)  
 $\begin{pmatrix} 1 & s_x^x & s_x^y & 0 \\ s_y^x & 1 & s_y^y & 0 \\ s_z^x & s_z^y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

$s_b^a$  is incr.  $a$ -axis in terms of  $b$ -axis.  $xy$ -shear defined to be  $s_z^x \neq 0$

**Commutativity:** Holds for **TT, SS, SR, RR** (only in 2D) with **T** = Translation, **R** = Rotation, **S** = Scaling  
**Concatenation:** Given sequence of Ops  $O_i$ , we can write the resulting Operation as one matrix:  $O_{tot} = \prod O_i$  evaluated from right to left.  
**Matrix Mult:** For  $A \cdot B = C: (C)_{i,j} = (A)_{i,:} \cdot (B)_{:,j}$   
**Inversion:**  $R^{-1} = R^T$ , Rigid Transforms = **T, R, S**

Change of coord System

$\begin{pmatrix} r_1 & r_2 & r_3 & t \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{pmatrix}$

Transforming a normal:  $n' = (M^{-1})^T n$ , Alternatively: (rot(axis))  $\times$  (shift by  $t$ )

**Projections:**  
 $M_{parallel} = \text{Diag}(1, 1, 0, 1), M_{perspective} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 0 \end{pmatrix}$  where  $d$  dist to camera (focal length)

**Quaternions**

Alternative approach for rotation. Similar to  $\mathbb{C}$ , define  $i^2 = j^2 = k^2 = -1, ijk = -1, ij = k, ji = -k, jk = i, kj = -i, ki = j, ik = -j$ . For  $q = a + bi + cj + dk$ , we have

- $\|q\| = \sqrt{a^2 + b^2 + c^2 + d^2}$
- $\bar{z} = a - bi - cj - dk$  and  $z^{-1} = \bar{z} / \|z\|$

Advantage: Efficient impl., Easy interpol., No Gimbal Lock

**Rotation by Quaternions**

Rotating a point  $p = \begin{pmatrix} x & y & z \end{pmatrix}^T$  around axis  $u = \begin{pmatrix} u_1 & u_2 & u_3 \end{pmatrix}$  by angle  $\theta$ .

- Convert  $p$  to quaternion  $p_Q = xi + yj + zk$
- Convert  $q$  to quaternion  $q'' = u_1i + u_2j + u_3k$ , normalize  $q' = q'' / \|q''\|$
- Rotate quaternion  $q = \cos \frac{\theta}{2} + q' \sin \frac{\theta}{2}$  and  $q^{-1} = \cos \frac{\theta}{2} - q' \sin \frac{\theta}{2}$
- Rotated point  $p' = qpq^{-1}$ . Convert to cartesian.

**2.4 Lighting & Shading**  
**Lighting:** Modelling physical interactions between materials and light sources  
**Shading:** Process of determining color of pixel  
**Solid angle:**  $\Omega = \frac{A}{r^2}$  steradians (where  $r$  radius)  
**Zenith, Azimuth:** Point  $\omega = (\theta, \phi)$  on unit sphere:  
 $\omega_x = \sin \theta \cos \phi, \omega_y = \sin \theta \sin \phi, \omega_z = \cos \theta$   
**Energy of photon:**  $\frac{hc}{\lambda} \left[ J = \frac{\text{kg m}^2}{\text{s}^2} \right]$

**Basic quantities of radiometry**

**Flux:**  $\Phi(A)$  total amount of energy passing through surface or space per unit time  $\left[ \frac{J}{s} = W \right]$   
**Luminous flux:**  $\int P(\lambda) V(\lambda) d\lambda$ . Perceived power of light weighted by human sensitiv. [lumen]  
**Irradiance:**  $E(x) = \frac{d\Phi(A)}{dA(x)}$  flux per unit area arriving at surface  $\left[ \frac{W}{m^2} \right]$ .  
**Radiosity:**  $B(x) = \frac{d\Phi(A)}{dA(x)}$  flux per unit area leaving a surface  $\left[ \frac{W}{m^2} \right]$   
**Radiant / Luminous Intensity:**  $I(\vec{\omega}) = \frac{d\Phi}{d\vec{\omega}}$  outgoing flux per solid angle  $\left[ \frac{W}{sr} \right]. \Phi = \int_{S^2} I(\vec{\omega}) d\vec{\omega}$   
**Radiance:**  $L(x, \vec{\omega}) = \frac{dI(\vec{\omega})}{dA(x)} = \frac{S^2}{d} \frac{d^2\Phi(A)}{dA(x) \cos \theta}$  flux per solid angle per perp. area = intensity per unit area

**Lamberts cosine law:** Irradiance at surface is proportional to cosine of angle of incident light and surface normal:  $E = (\Phi/A) \cos \theta$   
**Bidir. reflectance distr. func. (BRDF):** relation between incident radiance and diff. refl. radiance.

$$f_r(x, \vec{\omega}_i, \vec{\omega}_r) = \frac{dL_r(x, \vec{\omega}_r)}{dE_i(x, \vec{\omega}_i)} = \frac{dL_r(x, \vec{\omega}_r)}{L_i(x, \vec{\omega}_i) \cos \theta_i d\vec{\omega}_i}$$

**Reflection equation:** reflected radiance due to incident illumination from all directions (from BRDF):

$\int_{H^2} f_r(x, \vec{\omega}_i, \vec{\omega}_r) L_i(x, \vec{\omega}_i) \cos \theta_i d\vec{\omega}_i = L_r(x, \vec{\omega}_r)$   
**Types of reflections:** asc. in comp. complexity with BRDF =  $f_r$ :

1. **Ideal Diffuse:**  $f_r = c$
2. **Specular (Mirror):**  $f_r(\vec{\omega}_i, \vec{\omega}_r) = \delta(\vec{\omega}_r - \text{reflect}(\vec{\omega}_i))$
3. **Retro-reflective:**  $f_r$  depends on  $\vec{\omega}_i, \vec{\omega}_r$  but highly directional
4. **Glossy Specular:**  $f_r$  depends on  $\vec{\omega}_i, \vec{\omega}_r$

**Attenuation:**  $f_{att} = \frac{1}{d_L^2}$  due to spatial radiation. Used in OpenGL:  $f_{att} = \min\left(\frac{1}{c_1 + c_2 d_L + c_3 d_L^2}, 1\right)$

### Phong Illumination Model

Approximate specular reflection by cosine powers

$$I_{\lambda} = \underbrace{I_{a\lambda} k_a O_{d\lambda}}_{\text{Ambient}} + f_{att} \underbrace{I_{p\lambda} [k_d O_{d\lambda} (\mathbf{N} \cdot \mathbf{L})]}_{\text{Diffuse}} + \underbrace{k_s (\mathbf{R} \cdot \mathbf{V})^n}_{\approx \cos^n \beta \text{ Specular}}$$

- $I_{\lambda}$ : reflected intensity for  $\lambda$
- $I_{a\lambda}$ : Intensity of ambient light at  $\lambda$
- $k_a, 0 \leq k_a \leq 1$  Ambient reflection coefficient
- $O_{d\lambda}$  Object diffuse color at  $\lambda$
- $f_{att}$  : Attenuation factor, accounts for light intensity reduction with dist.
- $I_{p\lambda}$ : Intensity of the point light source at  $\lambda$
- $k_d, 0 \leq k_d \leq 1$  : Diffuse reflection coefficient, Controls how much of the incident light is diffusely reflected by the surface
- $\mathbf{N} \in \mathbb{R}^3$ : Surface normal
- $\mathbf{L} \in \mathbb{R}^3$ : Direction from the surface point to the light source
- $k_s, 0 \leq k_s \leq 1$ : Specular reflection coefficient, controls shininess of the surface
- $\mathbf{R} \in \mathbb{R}^3$ : Reflected light direction
- $\mathbf{V} \in \mathbb{R}^3$ : Viewing direction

**Extension:** Can be extended to multiple sources of light:  $I_{a\lambda} k_a + \sum_{1 \leq i \leq m} f_{att_i} I_{p_{\lambda_i}} [k_d (\mathbf{N} \cdot \mathbf{L}_i) + k_s (\mathbf{R}_i \cdot \mathbf{V})^n]$  and made faster with half-way vector:  $(\mathbf{N} \cdot \mathbf{H})^n$  where  $\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$

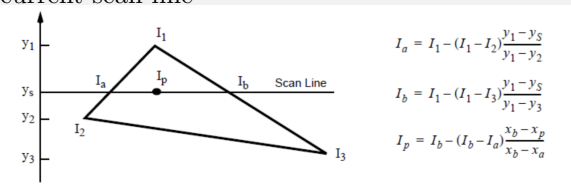
### 2.5 Shading Models

**Flat Shading (Screen Space):** one color per primitive, works when light-sources is at inf. or primitive is very small.

### Gouraud Shading (Screen Space)

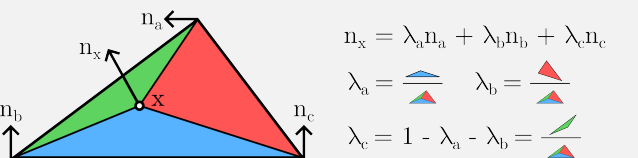
Lin.interpol. vertex intensities

1. **Calculate face normals**
2. **Calculate vertex normals:** For each vertex  $v$  compute  $N_v = (\sum_{i=1}^N N_i) \left( \left| \sum_{i=1}^N N_i \right| \right)^{-1}$ ,  $N_i$  are adjacent normals to  $V$
3. **Evaluate illumination model:** For each vertex
4. **Interpolate vertex colors:** Bilinearly on the current scan line



Problems with scan line interpolation are perspective distortion, orientation dependence, shared vertices. Quality depends on size of primitives.

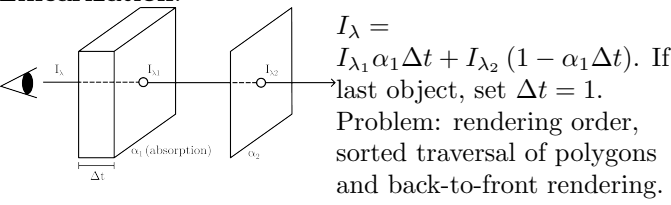
### Phong Shading (Object Space)



**Properties:** Lagrange:  $x = a \Rightarrow n_x = n_a$   
**Partition of unity:**  $\lambda_a + \lambda_b + \lambda_c = 1$   
**Reproduction:**  $\lambda_a \cdot a + \lambda_b \cdot b + \lambda_c \cdot c = x$ .  
**Problems:** normal not defined / representative.

**$\alpha$ -blending:** Achieves simple transparency. Perceived intensity  $I_{\lambda} = I'_{\lambda_1} + I'_{\lambda_2}$  where  $I'_{\lambda_1}$  is emission of  $P_1$  and  $I'_{\lambda_2}$  is intensity filtered by  $P_1$ . We model it as follows:  $I_{\lambda} = I_{\lambda_1} \alpha_1 \Delta t + I_{\lambda_2} e^{-\alpha_1 \Delta t}$  where  $\alpha$  absorption,  $\Delta t$  thickness.

**Linearization:**



Back-to-front order not always clear, resort to depth peeling, multiple passes, each pass renders next closest fragment. We need more advanced lighting models for

metal objects (Cook-Torrence), replacing specular term, with reflection from micro facets, self-shadowing...

### 2.6 Geometry & Textures

**Considerations:** Storage, acquisition of shapes, creation of shapes, editing shapes, rendering shapes. Explicit repr. can easily model complex shapes, and sample points, but take lots of storage.

- **Subdivision surface:** Define surfaces by primitives, use recursive algorithm for refining
  - **Point set surfaces:** Store points as surface
  - **Polygonal meshes:** Explicit set of vertices with position, possibly with additional information.
- Intersection of polygons: either empty, vertex, edge.

Implicit repr. easily test inside / outside, compact storage, but sampling expensive, hard to model **Implicit surfaces:** surface = zeros of a function

We need to store textures as bitmaps, hence parameterizing complex surfaces.

**Manifolds:** surface homeomorphic to disk, (1) most 2 faces share an edge, (2) conn. (half-)ring of faces

**Mesh data structures:** Locations, how vertices are connected, attributes such as normals, color etc. Must support rendering, geometry queries, modifications. E.g.

- Triangle list (list of 3 points, redundant, e.g. STL).
- Indexed Face set (array of vertices + list of indices, e.g. OBJ, OFF, WRL, costly queries, modifications)

**Bandlimiting:** Restrict amplitude of spectrum to zero for frequency beyond cut-off frequency.

**Anti-aliasing filters:**

- Gaussian (low-pass, separable). Closer pixels have higher weight
- Sinc  $S_{\omega_c}(x, y) = \frac{\omega_c \text{sinc}(\frac{\omega_c x}{\pi})}{\pi} \times \frac{\omega_c \text{sinc}(\frac{\omega_c y}{\pi})}{\pi}$  ideal lowpass filter, IIR filter.  $\omega_c$  cutoff freq. Hard to implement, has infinite impulse resposns.
- B-Spline: Allow for locally adaptive smoothing, adapt size of kernel to signal chars.

**Magnification:** for pixels mapping to area larger than pixel (jaggies), use bilinear interpolation.

**Mipmapping:** Store texture at multiple resolutions, choose based on projected size of triangle (far away  $\rightarrow$  lower res, interpol. possible). Avoids aliasing, improves efficiency, higher storage overhead. (minification)

**Supersampling:** Multiple color samples per pixel, final color is average - different patterns of samples possible (uniform, jittering, stochastic, Poisson).

**Light map:** Simulate effect of local light source. Can be pre-computed and dynamically adapted Environment map: Mirror environment with imaginary sphere or cube for reflective objects

**Bump map:** Perturb surface normal according to texture, represents small-scale geometry. Limitations: no bumps on silhouette, no self-occlusions, no self-shadowing. Height stored as grayscale textures.

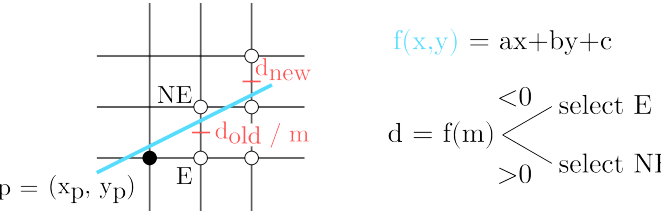
**Procedural texture:** Generate texture from noise (Perlin, Gabor) from Guassian pyramid of noise and summing layers with weights.

**Perspective interpolation** in world-space can yield non-linear variation in screen-space. Optimal resampling filter is hence spatially variant.

### 2.7 Scan Conversion

Generate discrete pixel values - approximate with finit amount.

**Bresenham line:** choose closest pixel at each intersection. Fast decision, criterion based on midpoint  $m$  and intersection point  $q$ . After computing first value  $d$ , we only need addition, bitshifts.  $d_{\text{new}} = d_{\text{old}} + a$



**Implementation:** `def bhmline(x0, y0, x1, y1):dx = x1 - x0;dy = y1 - y0;d = 2 * dy - dx;incE = 2 * dy;incNE = 2 * (dy - dx);x = x0;y = y0;writepixels(x, y);while x ≤ x1: if d ≤ 0:d += incE;else: d += incNE;y += 1;x += 1;writepixels(x, y);`

Scan conversion of polygons:

1. Calculate intersections on scan line
2. Sort intersection points by ascending  $x$ -coords.
3. Fill spans between two consecutive intersection points if parity in interval is odd (so inside)

**Implementation:** `def leftedgescan(xmin, ymin, xmax, ymax):x = xmin;;num = xmax - xmin;denom = ymax - ymin;inc = denom;for y in range(ymin, ymax + 1): writepixels(x, y);inc+= num;if inc > denom : x+= 1;inc-= denom`

### 2.8 Bézier Curves & Splines

**Disadvantages of Bézier:** global support of basis functions, new control pts yields higher degree,  $C^n$  continuity between segments of Bézier-Curves, imposes constraints.  $C^1$ :  $P_n^{(i)} = P_0^{(i+1)}$ ,  $C^2$ :  $P_{n-1}^{(i)} - P_n^{(i)} = k \cdot (P_1^{(i+1)} - P_0^{(i+1)})$

### Bézier Curves

$x(t) = b_0B_0^n(t) + \dots + b_nB_n^n(t)$  where  $B_i^n$  are the Bernstein polyn.  $\binom{n}{i} = \frac{n!}{i!(n-i)!} : B_i^n(t) = \binom{n}{i}t^i(1-t)^{n-i}$  if  $i \neq 0$  else 0 (partition of unity, pos. definite, recursion, symmetry). Coefficients  $b_0, \dots, b_n$  are called control- / Bézier-points, together define the control polygon.

Degree is highest order of polyn., order is deg. + 1. Bézier-Curves have affine invariance (affine transf. of curve accompl. by affine transf. of control pts.). Curve lies in convex hull of control polygon  $\text{conv}(P) := \{\sum_{i=1}^n \lambda_i p_i \mid \lambda_i \geq 0, \sum_{i=1}^n \lambda_i = 1\}$ . Control polyg. gives rough sketch of curve. Since  $B_0^n(0) = B_n^n(1) = 1$ , interpol. endpoints  $b_0, b_n$ . Max. number of intersect. of line with curve is  $\leq$  number of intersect. with control polygon. Bézier curves are special cases of B-spline curves.

### B-Spline functions

B-Spline curve  $s(u)$  built from piecewise polyn. bases  $s(u) = \sum_{i=0}^k d_i N_i^n(u)$  Coefficients  $d_i$  are called "de Boor" pts. Bases are piecewise, recursively def. polyn. over sequence of knots  $u_0 < u_1 < u_2 < \dots$  defined by knot vec.  $T = u = (u_0 \dots u_{k+n+1})$  Partition of unity  $\sum_i N_i^n(u) \equiv 1$ , positivity  $N_i^n(u) \geq 0$ , compact support  $N_i^n(u) = 0, \forall u \notin [u_i, u_{i+n+1}]$ , continuity  $N_i^n$  is  $(n-1)$  times cont. differentiable.

**deCasteljau:** Compute a triangular representation, successively interpolate, "corner cutting" with control points  $b_0, b_1, b_2$

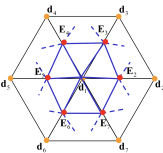
$$\begin{matrix} b_0 & & & b_0^1(t) = (1-t)b_0 + tb_1 \\ b_1 & b_1^1 & & b_1^1(t) = (1-t)b_1 + tb_2 \\ b_2 & b_1^1 & b_2^2 & \\ b_3 & b_2^1 & b_1^2 & b_0^3 & b_0^2(t) = (1-t)b_0^1(t) + tb_1^1(t) \end{matrix}$$

**deBoor:** generalizes deCasteljau, evaluate Bspline of degree  $n$  at  $u$ , set  $d_i^0 = d_i$ , finally  $d_0^n = s(u)$   $d_i^k = (1 - a_i^k) d_{i-1}^{k-1} + a_i^k d_{i+1}^{k-1}, a_i^k = \frac{u - u_{i+k-1}}{u_{i+n} - u_{i+k-1}}$   $a_i^k$  vanishes outside of  $[u_{i+k-1}, u_{i+n}]$

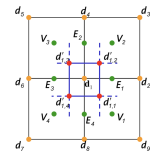
**Implementation:** `def deBoor(k: int, x: int, t, c, p: int):d = [c[j] + k - p] for j in range(0, p + 1)];for r in range(1, p + 1):for j in range(p, r - 1, -1):alpha = (x - t[j] + k - p) / (t[j] + 1 + k - r] - t[j + k - p]);d[j] = (1.0 - alpha) * d[j - 1] + alpha * d[j];return d[p]`

### 2.9 Subdivision Surfaces

Generalization of spline curves / surfaces allowing arbitrary control meshes using successive refinement (subdivision), converging to smooth limit surfaces, connecting splines and meshes.



**Loop Subdivision:**  
 $E_i = (\frac{3}{8}) \cdot (d_1 + d_i) + (\frac{1}{8}) \cdot (d_{i-1} + d_{i+1})$   
 $d_{1'} = \alpha_n \cdot d_1 + \frac{1-\alpha_n}{n} \cdot \sum_{j=2}^{n+1} (d_j)$   
 $\alpha_n = \frac{3}{8} + (\frac{3}{8} + \frac{1}{4n} \cos(2\pi))$



**Doo-Sabin Subdivision:**  
 $V_2 = \frac{1}{n} \sum_{j=1}^n d_j, E_i = \frac{1}{2} (d_1 + d_{2i})$   
 $d_{(1,j)'} = \frac{1}{4} (d_1 + E_j + E_{j-1} + V_j)$

**Properties:** Both: Appr, Loop:primal, works on triangles,  $C_1$  cont. (extra-ordinary points, valence  $\neq 6$ ),  $C_2$  cont. else ;Doo-Sabin:dual, works on arbitrary polygon meshes,  $C_1$  cont.,

**Classification:** faces are split (**primal**)vs vertcies are split (**dual**); new points are an interpolation of control points (**Interpolating**) vs controls points are moved, and new points are added inbetween (**Approximating**)

### 2.10 Visibility & Shadows

**Scan Line Shadow Calculation:** The edges of possibly shadow-casting polygons are projected onto all polygons in the direction of the light flux that are intersected by the current scan line.  $\mathcal{O}(n^2)$  **Painter's algorithm:** Render objects from furthest to nearest. Issues with cyclic overlaps & intersec.

**Z-Buffer:** store depth to nearest obj for each px. Initialize to  $\infty$ , then iter. over polygons and update z-buffer. Limited resolution, non-linear, place far from camera Planar shadows: draw projection of obj. on ground. No self-shadows, no shadows on other objects, curved surfaces.

**Projective texture shadows:** Separate obstacles and receivers, compute b/w img. of of obst. from light, use as projective texture. Need to specify obstacle & receiver, no self-shadows.

## Shadow maps

Compute depths from light & camera. For each px on camera plane, compute point in world coords., project onto light plane, compare  $d(\mathbf{x}_L)$  (dist. from light source to projected point on light plane) to  $z_L$  (dist light source to  $x_L$ ). If  $d(\mathbf{x}_L) < z_L$ , then  $x$  in shadow. Bias needed to avoid self-shadowing ( $d(x_L) + b < z_L$ ). Point to shadow can be outside field of view of (use cubical shadow map or spot lights). Aliasing due to undersampling shadow map (filter result of depth test).

**Shadow volumes:** Explicitly represent volume of space in shadow. If polygon in volume, it is in shadow. Shoot ray from camera, increment each time boundary of shadow volume is intersected. If counter  $> 0$ , primitive is in shadow. Optimization: use silhouette edges only. Introduces a lot of new geometry, expensive to rasterize long skinny triangles. Objects must be watertight for silhouette optimizations. Rasterization of polygons must

not overlap & not have a gap.

## 2.11 Ray Tracing

### Ray Tracing

Send rays into scene to determine color of px. On obj. hit, send mult. rays (diffused, reflected, refracted) further until we hit light source, or reach some amount of bounces. Figure out whether point in shadow by shooting rays to all light sources. For anti-aliasing, use mult. rays per px. (Pipel.: Ray Generation, Intersection, Shading)

**Ray-surface intersections:** Ray equation  $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ . Sphere:  $\|\mathbf{x} - \mathbf{c}\| - r^2 = 0$  where  $\mathbf{x}$  point of interest,  $\mathbf{c}$  center,  $r$  radius. Solve for  $t$ :  $\|\mathbf{o} + t\mathbf{d} - \mathbf{c}\|^2 - r^2 = 0$ . **Triangle:** Barycentric coords.  $\mathbf{x} = s_1\mathbf{p}_1 + s_2\mathbf{p}_2 + s_3\mathbf{p}_3$ . Intersect:  $(\mathbf{o} + t\mathbf{d} - \mathbf{p}_1) \cdot \mathbf{n} = 0$ . Using the following:  $\mathbf{n} = (\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)$  we get  $t = -\frac{(\mathbf{o} - \mathbf{p}_1) \cdot \mathbf{n}}{\mathbf{d} \cdot \mathbf{n}}$ . Now compute the coeffs.  $s_i$ . Test whether  $s_1 + s_2 + s_3 = 1$  and  $0 \leq s_i \leq 1$ . If so, inside triangle.

**Ray-tracing shading extensions:** Refraction, mult. lights, area lights for soft shadows, motion blur (sample objs, intersect in time), depth of field

**Cost:**  $O(N_x N_y N_o) = \#px \cdot \#objects$

**Accelerate:** Accelerate with less intersections, introduce uniform grids or space partitioning trees.

**Uniform grids:** Preprocess: compute bounding box, set grid res., rasterize objects, store refs. to objects. Traversal: incrementally rasterize rays - stop at intersection. Fast & easy, but non-adaptive to scene geometry.

**Space partitioning trees:** octree, kd-tree, bsp-tree

Another solution: bounding volume hierarchies (BVH)

1. Grayscale images range from 0 (black) to 255 (white).
2. Frequency: High (rapid changes in intensity or color)
3. Quotient Rule:  $(f/g)' = (f'g - fg')/(g^2)$
4. (255) need one byte. Thus RGB need 3 bytes per px
5. Butterfly (triangles, primal, interpol), Kobbelt (rect, primal, interpol), Catmull-Clark (rect, approx, primal)